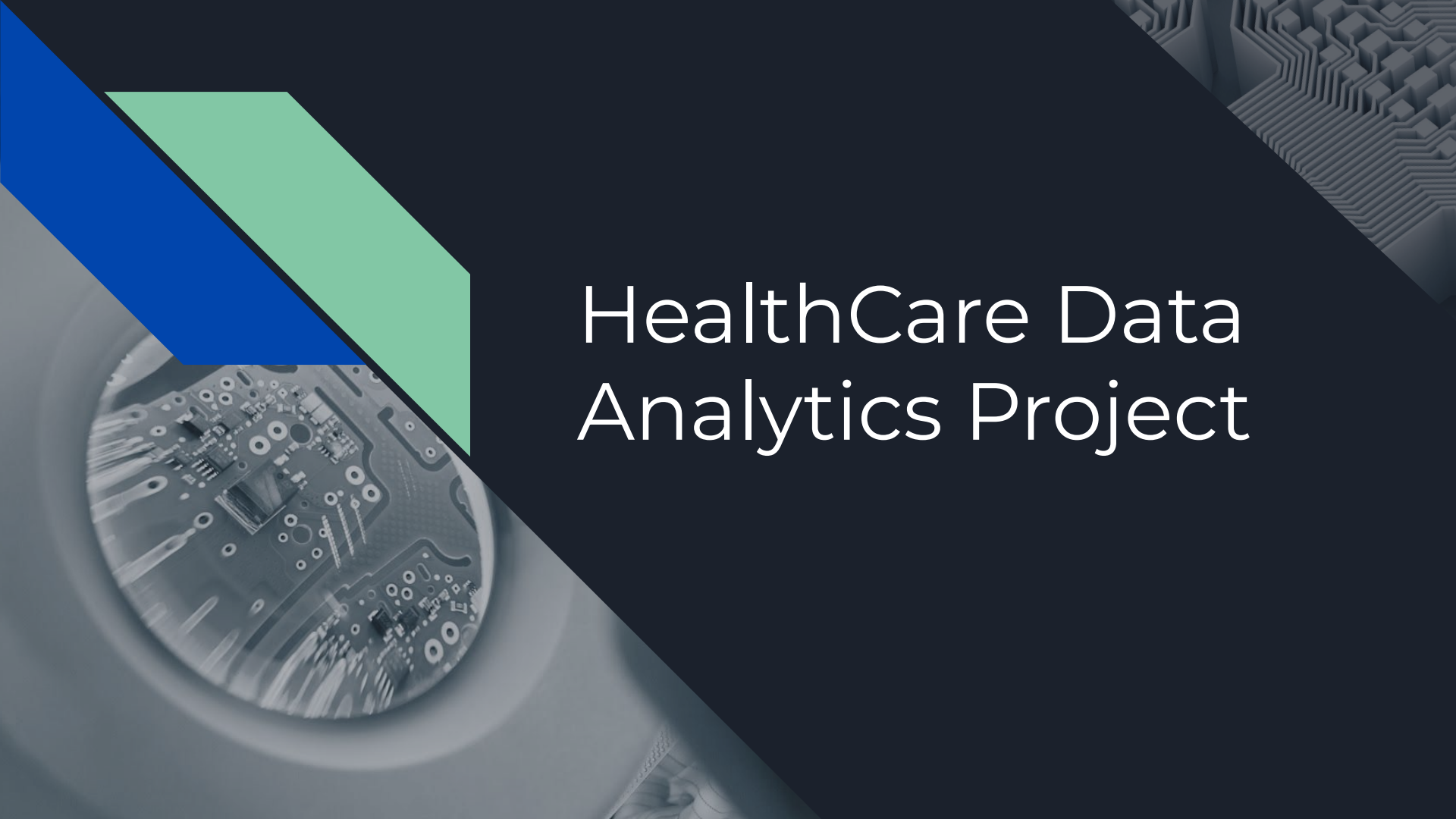


HealthCare Data Analytics Project

PROJECT SUMMARY

Problem Statement

Objective

Business Drivers

Expected Business Impact

Analysis Using Python

Conclusion

Project timeline: 8 Days

Overview

This project focuses on analyzing healthcare operational and financial data to generate actionable insights that support better decision-making. The analysis integrates multiple datasets related to **patients, doctors, appointments, treatments, and billing** to develop a comprehensive view of hospital performance.

The goal of the project is to move beyond raw data and transform it into **meaningful business intelligence** that helps identify patterns in patient behavior, evaluate doctor utilization, improve appointment efficiency, and understand key revenue drivers.

Using **SQL and Python** for data preparation and analysis, and **Power BI** for visualization, the project follows an end-to-end analytics lifecycle—from data understanding and cleaning to insight generation and executive-level reporting. The final output includes **KPIs, dashboards, and business recommendations** designed for healthcare stakeholders.

Overall, this project demonstrates the practical application of data analytics in a real-world healthcare context, highlighting how data-driven insights can improve **operational efficiency, patient experience, and financial performance**.

Problem Statement

Healthcare organizations manage large volumes of data related to patients, doctors, appointments, treatments, and billing.
However, this data is often **siloed and underutilized**, making it difficult for management to:

- Understand patient behavior and engagement
- Evaluate doctor performance objectively
- Identify revenue drivers and operational inefficiencies
- Make timely, data-driven decisions

As a result, hospitals may experience **inefficient resource utilization, higher no-show rates, and missed revenue opportunities**.

Project objective

The objective of this project is to perform end-to-end healthcare data analysis to:

- Analyze **patient demographics and visit patterns**
- Evaluate **doctor workload and performance**
- Identify **appointment trends and inefficiencies**
- Understand **treatment utilization and revenue contribution**
- Develop meaningful **KPIs and insights** to support operational and financial decision-making

Business Drivers

This analysis is driven by the need to:

- **Improve operational efficiency** by optimizing appointment scheduling and doctor utilization
- **Enhance patient experience** by identifying patterns behind no-shows and repeat visits
- **Increase revenue** by focusing on high-value treatments and patient segments
- **Support strategic planning** through data-backed performance metrics
- **Enable management decisions** with clear, actionable insights and dashboards

CSV Files

Total 5 CSVFile

1. Bill
2. Doctors
3. Patients
4. Treatments
5. Appointments

SQL Select Queries —----------

```
• select * from patients;
• select * from appointments;
• select * from billing;
• select * from doctors;
• select * from treatments;
```

SQL Queries

1. Total number of patients

```sql
SELECT count(*) as Patients_Count
FROM patients;
```

Result Grid | Filter Rows:

	Patients_Count
▸	50

2. Gender distribution

```sql
SELECT
    Gender,
    count(*)
FROM patients
GROUP BY 1;
```

Result Grid | Filter Rows:

	Gender	count(*)
▸	F	19
	M	31

SQL Queries

3. Age group analysis (0–18, 19–35, 36–50, 50+)

```sql
UPDATE patients
SET date_of_birth = STR_TO_DATE(date_of_birth, '%Y-%m-%d');

WITH CTE_1 as (
    SELECT
        *,
        round(datediff(now(), date_of_birth) / 365) as Age,
        case WHEN round(datediff(now(), date_of_birth) / 365) <= 18 THEN "0-18"
            WHEN round(datediff(now(), date_of_birth) / 365) <=35 THEN "19-35"
            WHEN round(datediff(now(), date_of_birth) / 365) <= 50 THEN "36-50"
            ELSE "50+"
        END as Age_Distribution
    FROM patients)

SELECT
    Age_Distribution,
    count(*)
FROM CTE_1
GROUP BY 1;
```

Result Grid | Filter Rows:

Age_Distribution	Patients
50+	19
36-50	13
19-35	18

SQL Queries

4. Top 5 Patients with highest number of visits

```sql
SELECT
    P.PATIENT_ID,
    concat(FIRST_NAME,' ', LAST_NAME) AS NAME,
    COUNT(APPOINTMENT_ID) AS NUMBER_OF_VISIT
FROM patients as p
LEFT JOIN appointments as a
ON a.patient_id = p.patient_id
GROUP BY 1,2
ORDER BY 3 DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

PATIENT_ID	NAME	NUMBER_OF_VISIT
P012	Laura Davis	10
P005	David Wilson	8
P049	David Moore	7
P016	Michael Taylor	7
P035	David Wilson	7

SQL Queries

5. Patients with highest medical spending

```sql
SELECT
    P.PATIENT_ID,
    concat(FIRST_NAME,' ', LAST_NAME) AS PATIENT_NAME,
    ROUND(SUM(AMOUNT)) AS TOTAL_AMOUNT
FROM PATIENTS AS P
INNER JOIN BILLING AS B
ON P.PATIENT_ID = B.PATIENT_ID
GROUP BY 1,2
ORDER BY 3 DESC
LIMIT 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

PATIENT_ID	PATIENT_NAME	TOTAL_AMOUNT
P012	Laura Davis	30053
P049	David Moore	23554
P016	Michael Taylor	22968

SQL Queries

6. Number of doctors by specialization

```sql
SELECT
    SPECIALIZATION,
    COUNT(*) AS TOTAL_DOCTORS
FROM DOCTORS
GROUP BY SPECIALIZATION;
```

Result Grid | Filter Rows: | Export

SPECIALIZATION	TOTAL_DOCTORS
Dermatology	3
Pediatrics	5
Oncology	2

SQL Queries

7. Appointments handled by each doctor

```sql
SELECT
    a.Doctor_id,
    concat(first_name,' ',last_name) as Doctor_Name,
    count(appointment_id) AS Appointment
FROM doctors as d
INNER JOIN appointments as a
ON d.doctor_id = a.doctor_id
GROUP BY 1,2
ORDER BY 3 DESC;
```

Result Grid | Filter Rows: | Export:

Doctor_id	Doctor_Name	Appointment
D005	Sarah Taylor	29
D001	David Taylor	25
D006	Alex Davis	24
D003	Jane Smith	22
D002	Jane Davis	21
D010	Linda Wilson	19
D009	Sarah Smith	17
D008	Linda Brown	16
D004	David Jones	14
D007	Robert Davis	13

SQL Queries

8. Revenue generated per doctor

```sql
SELECT
    d.doctor_id,
    concat(first_name,' ',last_name) as Doctor_Name,
    round(sum(amount)) as Total_Revenue
FROM appointments as a
LEFT JOIN billing as b
ON a.patient_id = b.patient_id
LEFT JOIN doctors as d
ON d.doctor_id = a.doctor_id
GROUP BY 1,2
ORDER BY 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conten

doctor_id	Doctor_Name	Total_Revenue
D007	Robert Davis	188583
D004	David Jones	191213
D009	Sarah Smith	214193
D008	Linda Brown	234358
D002	Jane Davis	264651
D003	Jane Smith	279957
D010	Linda Wilson	304846
D001	David Taylor	336852
D006	Alex Davis	339281
D005	Sarah Taylor	394357

SQL Queries

9. Appointments by month

```sql
WITH app_mon as (
SELECT *,
    monthname(appointment_date) as Month_
FROM appointments)

SELECT
    Month_,
    count(*)
FROM app_mon
GROUP BY Month_;
```

Result Grid | Filter Rows:

Month_	count(*)
August	15
June	18
September	11
July	16
April	25
May	19
March	19
January	20
November	17
February	14
December	12
October	14

SQL Queries

10. Most Common Treatment

```sql
SELECT
    Treatment_Type,
    count(*) as Count
FROM treatments
GROUP BY treatment_type;
```

Result Grid | Filter Rows:

Treatment_Type	Count
Chemotherapy	49
MRI	36
ECG	38
Physiotherapy	36
X-Ray	41

SQL Queries

11. Treatments by specialization

```sql
SELECT
    d.specialization as Specialiazation,
    t.Treatment_type as Treatment,
    count(*) as Count
FROM appointments as a
LEFT JOIN doctors as d
ON a.doctor_id = d.doctor_id
LEFT JOIN treatments as t
ON t.appointment_id = a.appointment_id
GROUP BY 1,2
ORDER BY 1,3 DESC;
```

Result Grid | Filter Rows: | Export:

Specialiazation	Treatment	Count
Dermatology	X-Ray	17
Dermatology	Chemotherapy	17
Dermatology	Physiotherapy	15
Dermatology	ECG	13
Dermatology	MRI	8
Oncology	ECG	9
Oncology	MRI	8
Oncology	Physiotherapy	6
Oncology	Chemotherapy	5
Oncology	X-Ray	4
Pediatrics	Chemotherapy	27
Pediatrics	MRI	20

SQL Queries

12. Revenue by treatment type

```sql
SELECT
    Treatment_Type,
    round(sum(Cost)) as Total_Revenue
FROM treatments
GROUP BY 1;
```

Result Grid | Filter Rows: | Export:

Treatment_Type	Total_Revenue
Chemotherapy	128856
MRI	116098
ECG	96224
Physiotherapy	99418
X-Ray	110654

SQL Queries

13. Patient lifetime value (total spend per patient)

```sql
SELECT
    p.Patient_ID,
    concat(p.first_name,' ',p.last_name) as Name,
    round(sum(Amount)) as Total_Spent
FROM patients as p
LEFT JOIN billing as b
ON p.patient_id = b.patient_id
GROUP BY 1,2;
```

Result Grid | Filter Rows: | Export:

Patient_ID	Name	Total_Spent
P001	David Williams	7719
P002	Emily Smith	5969
P003	Laura Jones	7937
P004	Michael Johnson	5363
P005	David Wilson	18610
P006	Linda Jones	NULL
P007	Alex Johnson	10734
P008	David Davis	3637
P009	Laura Davis	10557
P010	Michael Taylor	15929
P011	Emily Jones	7729
P012	Laura Davis	30053
P013	Laura Johnson	9308

SQL Queries

14. Which doctor specialization is most profitable?

```sql
SELECT
    Specialization,
    round(sum(Amount)) as Total_Amount
FROM doctors d
LEFT JOIN appointments as a
ON d.doctor_id = a.doctor_id
LEFT JOIN billing as b
ON a.patient_id = b.patient_id
GROUP BY 1;
```

Result Grid Filter Rows:

Specialization	Total_Amount
Dermatology	965567
Pediatrics	1289295
Oncology	493429

SQL Queries

15. Which age group generates the highest revenue?

```sql
WITH Age_group as (
SELECT
    *,
    round(datediff(now(),date_of_birth)/365) AS Age,
    CASE WHEN round(datediff(now(),date_of_birth)/365) <=18 THEN "<18"
        WHEN round(datediff(now(),date_of_birth)/365) <= 35 THEN "19-35"
        WHEN round(datediff(now(),date_of_birth)/365) <=50 THEN "36-50"
        ELSE ">50"
    END as Age_Distribution
FROM patients)

SELECT
    Age_Distribution,
    round(sum(amount)) as Total_Revenue
FROM Age_group as ag
LEFT JOIN billing as b
ON ag.patient_id = b.patient_id
GROUP BY 1
ORDER BY 2;
```

Result Grid | Filter Rows: | Export: | Wrap Ce

Age_Distribution	Total_Revenue
36-50	116809
>50	196492
19-35	237948

PYTHON

Analysis Using Python

Python is a powerful and versatile programming language widely used in data analytics due to its simplicity and readability. It allows efficient data manipulation, analysis, and visualization using libraries such as Pandas, NumPy, Matplotlib, and Seaborn.

In analytics projects, Python helps in data cleaning, feature engineering, exploratory data analysis, and generating meaningful insights.

Its strong community support and extensive libraries make it an essential tool for solving real-world business problems efficiently.

Import Libraries & Insert into DataFrame

```
[1]: import pandas as pd
     import numpy as np
     import matplotlib.pyplot as plt
     import seaborn as sns

     bill = pd.read_csv("billing.csv")
     doctors = pd.read_csv("doctors.csv")
     patients = pd.read_csv("patients.csv")
     treatments = pd.read_csv("treatments.csv")
     appointments = pd.read_csv("appointments.csv")
```

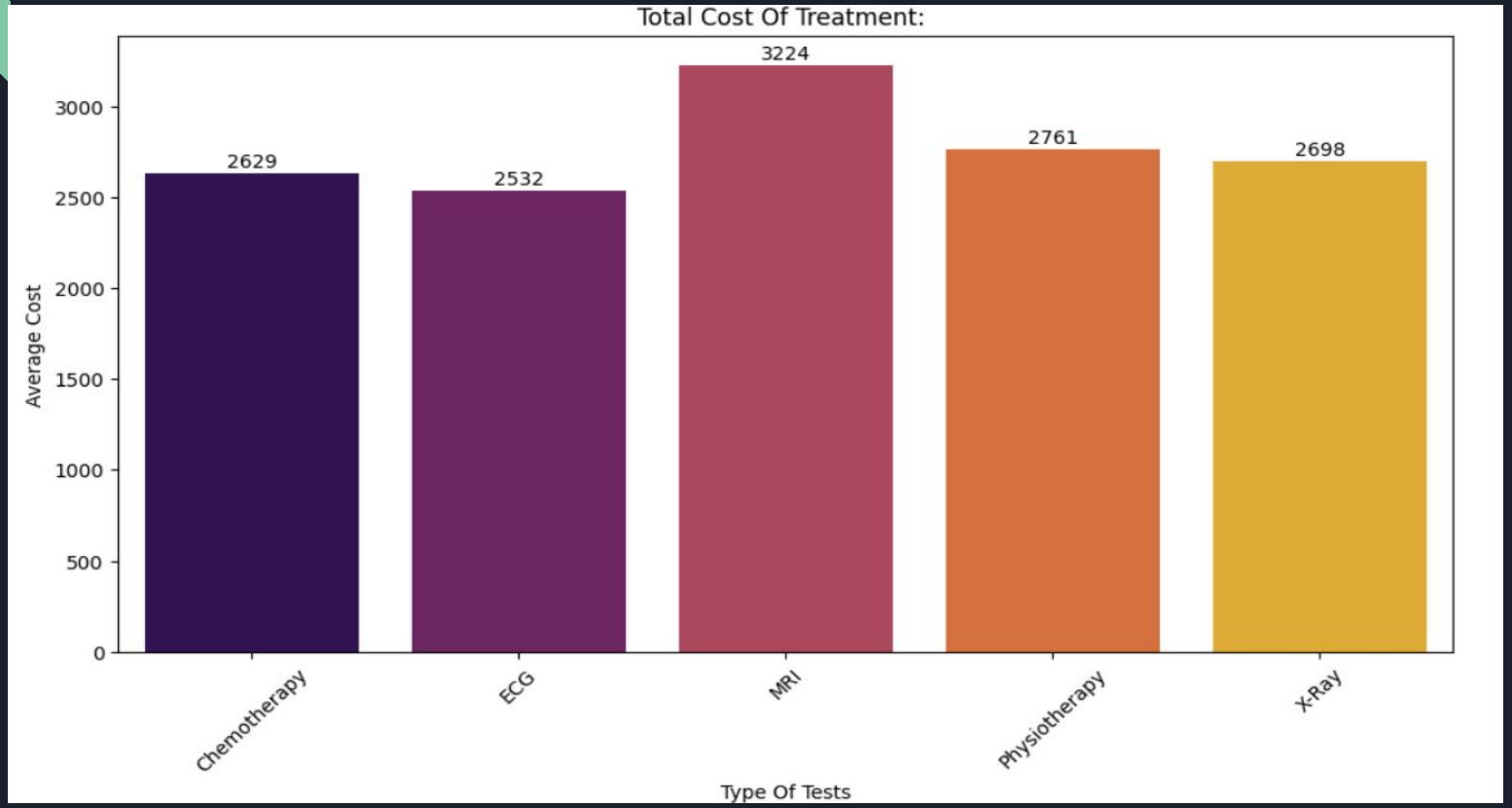
1. Code For Treatment Average Cost

```
[2]: treatment_average_cost = treatments.groupby("treatment_type")["cost"].mean().reset_index()
treatment_average_cost
```

[2]:

	treatment_type	cost
0	Chemotherapy	2629.707755
1	ECG	2532.216842
2	MRI	3224.948889
3	Physiotherapy	2761.613889
4	X-Ray	2698.870000

Average Price of Treatment

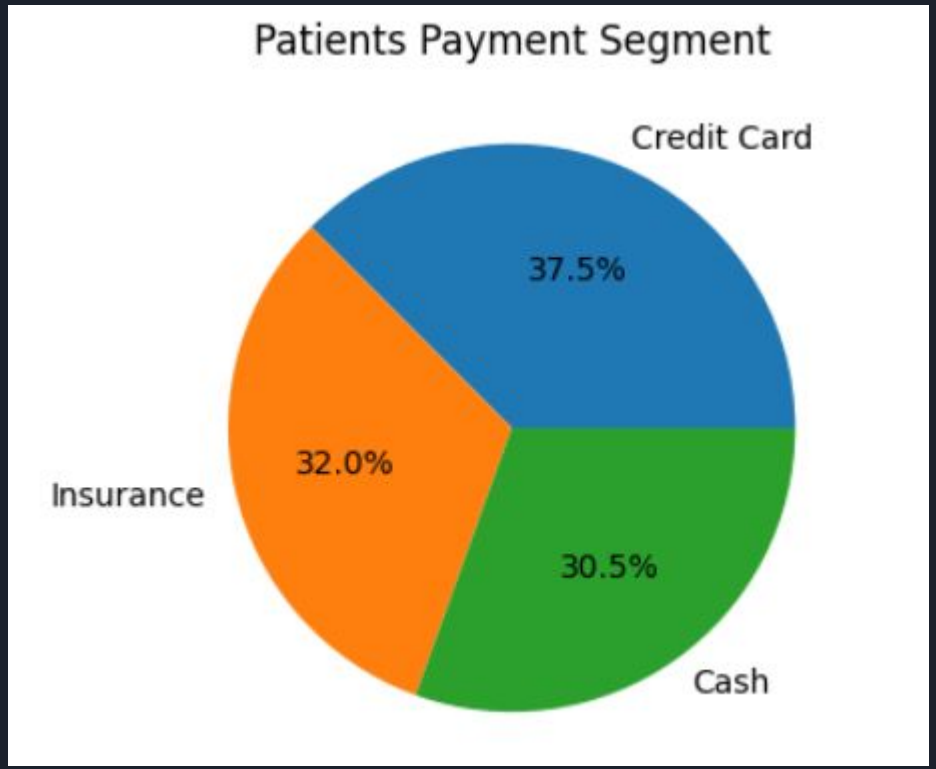


2. Code For Treatment Average Cost

```
[6]: plt.figure(figsize=(4,4))
     plt.pie(payment_method['Count'], labels= payment_method["payment_method"], autopct="%1.1f%%")
     plt.title("Patients Payment Segment")
     plt.show()
```

Result On Next Page

Payment Method

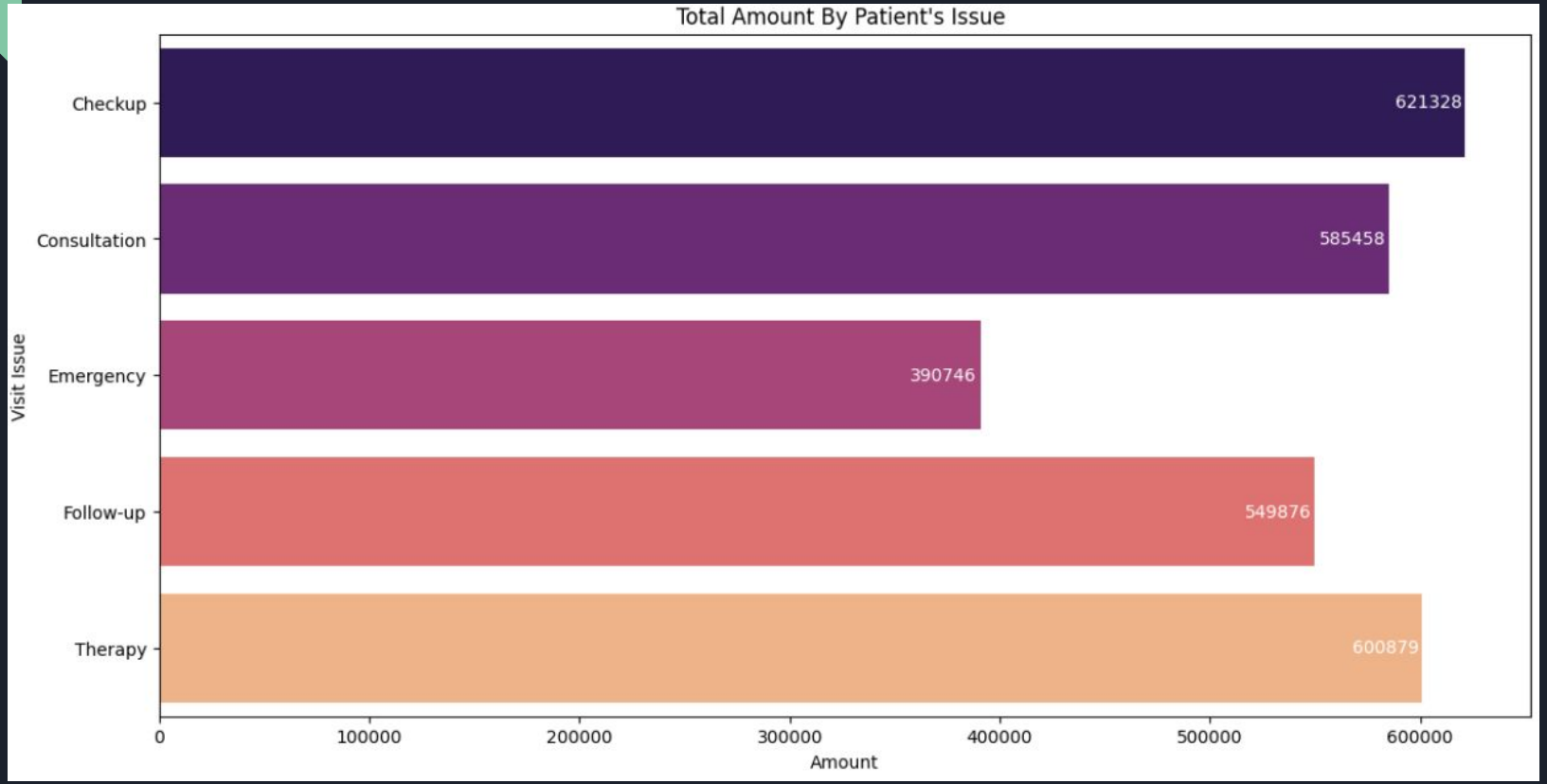


4. Total Amount By Patient Issue

```
plt.figure(figsize=(14,7))
ax = sns.barplot(data = bill_amt, x = "amount", y = "reason_for_visit", palette= "magma")
plt.title("Total Amount By Patient's Issue ")
plt.xlabel("Amount")
plt.ylabel("Visit Issue")
for container in ax.containers:
    ax.bar_label(container, fmt = "%d", padding= -40, color = "white")
plt.show()
```

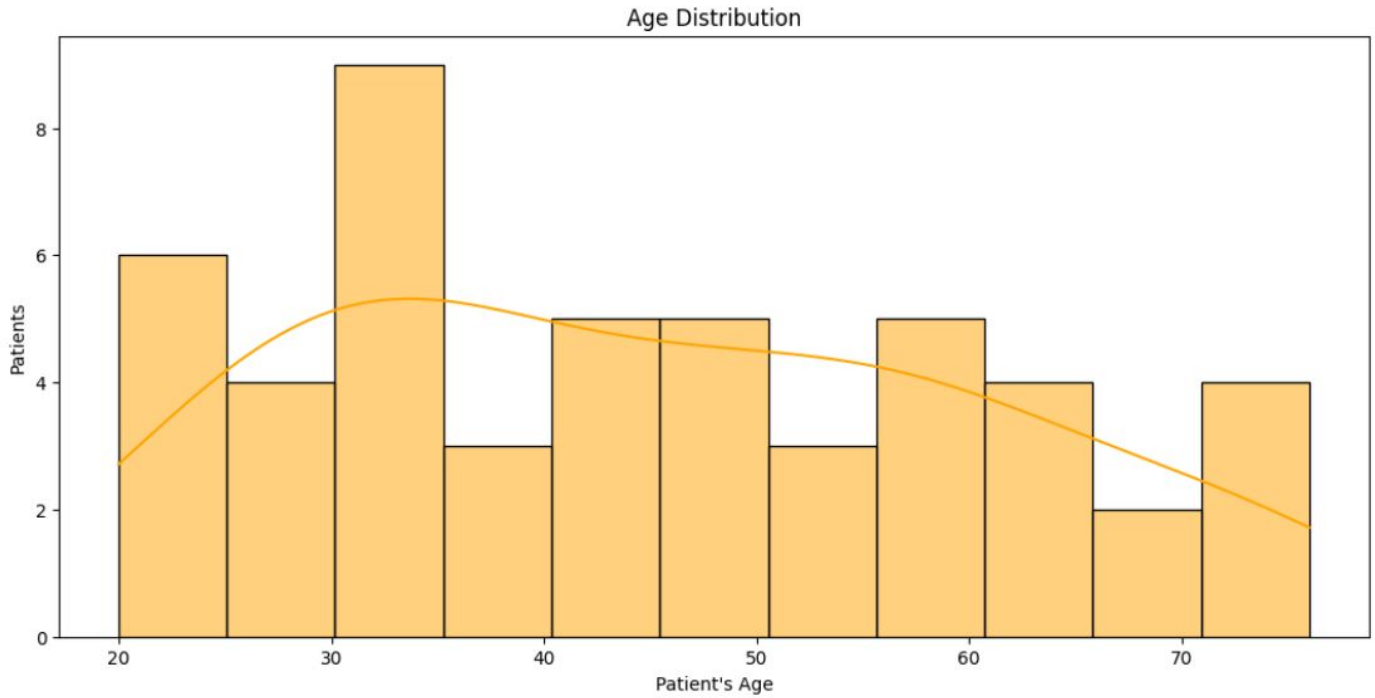
Result On Next Page

Total Revenue



4. Age Distribution

```
[13]: plt.figure(figsize=(13,6))
      sns.histplot(x = patients["Age"], bins = 11, kde = True, color = "orange")
      plt.title("Age Distribution")
      plt.xlabel("Patient's Age")
      plt.ylabel("Patients")
      plt.show()
```



Conclusion

This project was completed using both Python and SQL to ensure a strong end-to-end data analysis workflow. SQL was used to extract, filter, and aggregate structured data efficiently from the database, while Python was utilized for data cleaning, feature engineering, and visualization to generate meaningful insights. By combining querying capabilities with analytical and visualization tools, the project demonstrates the ability to handle real-world datasets, uncover patterns, and support data-driven decision-making effectively.